Task 4: Convolutional Neural Networks for Vision

Goal

In this task you will design, build, and train your own Convolutional Neural Network (CNN) in PyTorch. There is no step-by-step recipe: you choose the architecture and training setup, and your model must reach a target accuracy on the dataset you pick. By the end you should be able to:

- Build a CNN from standard PyTorch layers
- Explain why convolutions are well suited to images (local connectivity, weight sharing, translation equivariance)
- Diagnose and improve a vision model's performance

Background

Watch this lecture on convolutional layers before starting: `https://www.youtube.com/watch?v=pqIcoskUuWs`. Your notes on the lecture are part of the submission.

Code Structure

Use the same four files as Task 3. Most of your Task 3 code will carry over; the main work is in `model.py`.

- **main.py** – main orchestration
- **train.py** – training and evaluation
- **data.py** – data loading and pre-processing (update for your chosen dataset)
- **model.py** – your CNN architecture

Task

Pick **one** of the datasets below from `torchvision.datasets` (`https://docs.pytorch.org/vision/stable/datasets.html`) and train a CNN that reaches at least the target accuracy on the dataset's official test split.

Dataset	torchvision class	Target test accuracy
CIFAR-100	`CIFAR100`	40%
STL-10	`STL10`	55%
Oxford-IIIT Pet	`OxfordIIITPet`	25%

Rules

- The architecture must be defined by you in `model.py` using standard PyTorch building blocks (`nn.Conv2d`, `nn.BatchNorm2d`, pooling, `nn.Linear`, etc.).
- **No pretrained weights and no pre-built architectures** (e.g. `torchvision.models`, `timm`). You may take inspiration from published designs such as VGG or ResNet, but you must implement them yourself and train from scratch.
- Train only on the training split. Report accuracy on the official test split; never train or tune directly on it.
- Anything else is fair game: data augmentation, normalization, dropout, optimizers, learning-rate schedules, residual connections, and so on.

Running on Nautilus

Train your model on Nautilus. Include a screenshot showing your code running there for at least several epochs, with the training loss and accuracy visible.

Submission

Submit a single `.zip` file containing:

1. All four code files (`main.py`, `train.py`, `data.py`, `model.py`)
2. Your lecture notes
3. The Nautilus screenshot
4. A short write-up (half a page is fine) covering:
    - Your dataset and final test accuracy
    - A brief description of your architecture
    - One or two changes that most improved your accuracy, and why you think they helped
    - The random seed that you used to reach the reported accuracy

We will run your code to verify that it trains and reaches the reported accuracy.